

flint-lite — differences from POSIX

flint-lite presents an ordinary POSIX filesystem over NFSv4 and keeps its durable copy in an S3 bucket. Almost everything POSIX promises holds; a short list does not. This page enumerates that list, in the manner of CephFS’s “Differences from POSIX”, and says for each one whether it is refused loudly, lost quietly, already fixed, or irreducible. **It is a statement of behaviour, not a measured conformance score** — page 5 says exactly what has and has not been measured, and why the distinction matters.

flint-lite is not a FUSE-over-S3 filesystem read this first

A hub pod runs an NFS server over a **real ext4 volume**; clients mount it with the ordinary kernel NFS client. S3 is a **tier underneath** — a flush/evict/hydrate loop — not the store that answers the syscall. So the familiar object-storage complaints (no partial writes, no **mmap**, no locking, no atomic rename, a millisecond per **stat**) are **not** flint-lite’s. Page 2 lists what that buys.

Three kinds of difference — and none of them is consistency
Representation — state S3 cannot encode at all (hard links, sparseness, device nodes). **Asynchrony** — the bucket is a correct view of a *past* moment. **Divergence** — two authorities, local and remote, that can disagree. S3 has had strong read-after-write consistency since 2020 and flint publishes with compare-and-swap on top; consistency is the one thing that is *not* the residual.

How to read a row
Refused — the operation fails with an error the application sees. **Quiet** — it succeeds and something is lost or altered; the caller is not told. **Fixed** — was a defect, closed 2026-08-28, with a test. **Irreducible** — not a backlog item; no design on this architecture removes it. Quiet is the category that matters: it is the one an operator cannot discover.

The whole list, at a glance

Every difference on the following pages, and where it becomes visible

Category	At the mount	Through evict → hydrate	Through a DR restore from the bucket
Representation	Hard links refused; special files become regular files; two reserved names	Holds, except sparseness (a restored file is dense)	ctime and atime not carried; mtime at second granularity; sparseness lost; xattrs/ACLs have no front door to lose
Asynchrony	fsync means “on the volume”, not “in the bucket”	—	Restores to the last flush barrier; newer content is beyond the RPO and is counted, not silently dropped
Divergence	—	Foreign bucket writes and a second hub are arbitrated, not merged	The manifest lane and the bucket-sweep lane do not have equal fidelity

The single most useful sentence about this design: **the local volume converts S3 from a filesystem into a replication target**. That removes the entire syscall-shaped problem list. What it cannot remove is that the target cannot express everything the source holds, that replication is asynchronous, and that there are now two authorities instead of one.

What POSIX holds — and why that is not obvious

Most documents of this kind for object-backed storage open with a long list of things that do not work. That list belongs to filesystems where S3 answers the syscall. flint-lite’s syscalls are answered by ext4, so the list is short and the guarantees below are ordinary. They are stated explicitly because a reader who has used `s3fs`, `goofys` or `mount-s3` will reasonably assume otherwise.

Guarantees that hold in full

Left: what POSIX promises. Right: what an object-storage-backed mount does instead — the reason each row is worth stating.

Guarantee	flint-lite	A filesystem where S3 answers the syscall
Partial writes	Ordinary <code>pwrite</code> at any offset; the write lands in the local file	Objects are immutable — a one-byte write reads, modifies and re-uploads the whole object
Append	Ordinary <code>O_APPEND</code>	Not expressible; the object is rewritten
Atomic rename	<code>rename(2)</code> is ext4’s, atomic, including rename-over	COPY + DELETE: not atomic, and a directory rename is O(objects)
Byte-range locks	NFSv4 locking, state persisted across restart	No locking primitive exists; coordination must be bolted on outside
Unlinked-but-open	Data survives until the last descriptor closes	DELETE is immediate
mmap	Shared mappings work as the kernel NFS client implements them	Each dirty page implies an object rewrite
Directory reads	Stable <code>readdir</code> cookies with a verifier	Pagination tokens are not seekable directory offsets
Metadata cost	<code>chmod</code> , <code>chown</code> , <code>utimes</code> are local inode writes	<code>chmod</code> on a 1 GiB object is a 1 GiB server-side copy
Latency	Local-filesystem latency behind one network hop	A round trip per metadata operation; <code>ls -lR</code> becomes thousands
Empty directories	Exist, and survive a DR restore — the manifest carries directories as entries	Cannot exist: no object, no directory

Where the trade actually lands. These guarantees are bought with a real volume, and that volume is also what creates the divergence class on page 4: a second authority that can disagree with the bucket. The same architecture choice pays for page 2 and bills for page 4. flint’s other front end, **flint-passthrough**, makes the opposite choice and inherits the right-hand column in full.

Representation — state that S3 cannot encode

These are the true POSIX differences. Each one exists because the object store has no way to express the property, not because of a bug, and each fails identically whether the store is strongly or eventually consistent.

● refused — the caller is told ● quiet — the caller is not told ● fixed 2026-08-28 ● irreducible

Property	What flint-lite does	Why, and what it means for you
● Hard links	LINK is refused with NFS4ERR_NOTSUPP while the tier is on, and FATTR4_LINK_SUPPORT now reports FALSE so clients ask before they try.	A bucket key names one object; a hard link is one content under two names. tar , pax , rsync -H and cp -a read the support bit and fall back to copying, which is why reporting it truthfully matters more than the refusal itself.
● Pre-existing hard links	A pair that arrives by import, or predates the tier being switched on, is now safe: removing one name no longer deletes the shared object or drops the surviving name's rows.	Until 2026-08-28 it did, and if the inode had been evicted the surviving name became a permanent 0-byte file that read as EOF under NFS4_OK . Closed, with a test.
● Special files	CREATE of a socket, FIFO, block or character device makes a regular file and returns NFS4_OK .	Deliberate: mknod for devices needs root, and an export must not let callers dereference host devices. But mkfifo(3) succeeds with the wrong type and nothing says so — the one row here where the honest answer would be an error.
● Sparseness	A restored file is dense: holes come back as real zero bytes, and st_blocks is not carried.	Documented by the tier as declined. Note the capacity consequence: a 1 TiB sparse file restores as 1 TiB of zeros onto a volume sized for the working set.
● ctime	Not carried, and not restorable. A hydrated or DR-restored file's ctime is the restore time.	ctime is settable through no POSIX interface, and every path that restores content opens the file for writing. This one is documented rather than fixed because nothing on this architecture can fix it.
● atime	The tier has no atime concept; its eviction LRU is an approximation of one.	Arguably correct to decline: carrying atime faithfully means a metadata write per read.
● mtime precision	Carried at second granularity through a DR restore. Evict → hydrate preserves it exactly.	POSIX carries nanoseconds. Build systems and rsync compare mtimes, so a restored tree can produce spurious or missed rebuilds.
● Filenames	A POSIX name is any byte string without NUL or /. An S3 key is UTF-8, at most 1024 bytes.	Names outside that set cannot be published. This is the least-explored corner of the design and the one most likely to still hold surprises.
● Reserved names	Two directory names are reserved throughout the tree: .flint (tier control objects) and .flint-nfs (the server's filehandle key and state database).	A real namespace carve-out. Until 2026-08-28 the second was not reserved <i>from the tier</i> , so the server's own key was publishable and evictable — truncating it would have made every filehandle in the volume permanently stale.
● xattrs and ACLs	No extended-attribute operations exist in the server at all; GETATTR(acl) returns an empty ACL and setting one is refused with ATTRNOTSUPP .	These are not tier losses. There is no front door for the tier to lose anything through, so they are absent server features rather than round-trip gaps — a much larger project, planned separately.

Asynchrony and divergence — the two costs that are not representation

Neither of these is a missing feature, and neither is fixable by carrying more metadata. They are properties of putting a durable copy somewhere else, asynchronously.

Asynchrony — the durability boundary

The bucket is a correct view of a past moment

fsync means “on the volume”, not “in the bucket”. Data is durable locally when the call returns; it reaches S3 at the next flush barrier. A DR restore is therefore POSIX-faithful to a point in the past, not to the moment of loss.

This is not a defect and not a consistency problem — S3 is telling the truth. It is the RPO. The manifest states its own loss bound: files present locally but never published are counted in **beyond_rpo**, so the bucket can say how far behind it is *from the bucket alone*.

The limit worth understanding. NFSv4 has no wire mechanism for “this write succeeded and is not yet durable elsewhere”. Every surface that could report the lag — status, metrics, an operator condition — is control plane. The application that called write(2) never sees it. No design removes this; it is a property of the protocol, and it is why the honest posture is to make the lag **visible and bounded** rather than to claim it away.

Closing it entirely would mean flushing synchronously to S3 on every **fsync**, at object-store latency per call. That is a different product.

Divergence — two authorities

Created by the local volume, not removed by it

A filesystem where S3 answers the syscall has one source of truth. flint-lite has two, and they can disagree: someone writes into the prefix with **aws s3 cp**; a second hub adopts the same prefix; a crash lands between a local truncate and its durable marker.

The tier’s epoch lease, tombstones, flush intents and HEAD-based 412 arbitration exist for exactly this. They are not consistency machinery — they are **divergence** machinery, and they are the bill for page 2.

Two lanes, two fidelities. A DR restore driven by the manifest carries mode, uid, gid, mtime, directories and symlink targets. A restore that sweeps a bucket with no manifest cannot — those live only in the manifest. The gap between the two lanes has not been measured.

Consistency is not the residual. S3 has had strong read-after-write consistency, including for listings, since December 2020, and flint does not rely on it: every publish is a compare-and-swap under If-Match. The real risk is an S3-compatible store that accepts conditional headers and ignores them, which would silently degrade arbitration to last-writer-wins. Since 2026-08-28 the hub probes for this at startup and refuses to serve a store that fails it.

Status — and what has not been measured

The most important thing on this page is the second half. A differences list is only as good as the instrument that would have caught an omission, and that instrument does not exist yet for flint-lite.

Closed 2026-08-28

Each with a test observed red before the fix and green after

Server’s own directory	.flint-nfs was publishable and evictable; truncating fh.key would have made every filehandle permanently stale
Eviction race	The evictor opened a client-writable path by name with no symlink guard and no identity re-check — a substituted inode, or a planted symlink, could be truncated instead
Hard-link removal	Unlinking one name deleted the shared object and dropped the surviving name’s rows
Re-key	A re-key deleted a bucket object another live row still cited
Failed hydration	A failed restore permanently rewrote the file’s mtime
Restore permissions	A DR restore stripped setuid and setgid from every file it restored
Capability honesty	LINK_SUPPORT advertised hard links the server refuses
Store conformance	The hub trusted, without checking, that its object store enforces conditional writes

What is not measured

Stated plainly, because the alternative is a number that is not true

There is no POSIX conformance number for flint-lite. The repository runs pjdfstest — 8,798 assertions, differenced against a knfsd control — and it runs with **the S3 tier switched off**. In that configuration LINK is **never refused and not one object is ever written**, so no round-trip behaviour on this page is exercised by it.

The recorded figures — 645 flint-only failures in the default posture, 239 with permission enforcement on, against 149 for the knfsd control — are a conformance result for the hub **as an NFS server**. They are the floor for flint-lite, not a measurement of it.

Two arms would produce one. First, the same suite with the tier live, expecting the hard-link tests to newly fail as a recorded deliberate delta. Second — and this is the one that does not exist in any form — a round-trip fidelity differential: build a deliberately hostile tree, fingerprint every attribute, force it through evict→hydrate and through destroy→restore, and diff. pjdfstest structurally cannot do this: each of its tests creates, asserts and removes inside one test body, leaving no seam where a flush barrier could land between the write and the assertion.

Until those run, every claim on pages 3 and 4 is a verified reading of the code and none of them is a measured bound.

Method: a six-lens code audit (by NFSv4 attribute, by operation, by POSIX semantics, by round-trip stage, by name encoding, by file identity), each lens’s findings re-read by an adversarial refuter, then the high-severity undocumented findings put to three independent skeptics each; 48 raw findings, 6 confirmed. Every fix listed above carries a test that was observed failing with the fix removed. Known limits of that audit: its own adversarial pass killed only one finding, several results were duplicated across lenses, eight tier modules were never opened, and no formal model covers re-keying or link counts. Verified against code at **b871595e** plus the tier fixes of 2026-08-28; the fixes are not yet committed. Companion documents: **docs/plans/posix-fidelity-measurement-plan.md** (the two arms) and **docs/plans/xattr-acl-support-plan.md**.